Duet: Orchestrating Buffer Management in Cloud-Native Databases

Full Research Paper

Anonymous Author(s)

ABSTRACT

Cloud-native databases increasingly disaggregate compute, memory, and storage across asymmetric hardware connected by fast RDMA/CXL-class interconnects. This shift exposes the traditional monolithic buffer manager as a critical bottleneck. We show that coupled buffer management—whether placed at Compute Nodes (CNs) or Memory Nodes (MNs)—mismatches the hardware: MN-centric designs overload limited MN CPUs with elastic replacement work, while CN-centric designs couple eviction with dirty-page write-back, causing capacity cliffs and data-tromboning bandwidth bottlenecks.

We present Duet, a disaggregated buffer manager that separates CPU-intensive replacement policy and foreground swap-in from bandwidth-bound background swap-out. Duet uses two engines: **Primo**, running on CNs, executes policy and foreground swap-in, makes fine-grained eviction decisions, non-blockingly offloads dirty pages to fixed-address remote slots, and later reserves selected staged pages for eviction; **Secondo**, running on MNs, consumes the resulting queue and asynchronously writes the reserved pages to storage. MN-resident metadata coordinates the two engines. This split avoids dirty-page bounce-back through CNs and reduces interference between background write-back and foreground execution. Across three workloads and five cache settings on a real RDMA cluster, Duet improves throughput by approximately 34% and reduces latency by approximately 29% on average relative to the better-performing baseline at each setting.

KEYWORDS

cloud-native databases, buffer management, RDMA, disaggregated architecture, remote memory

ACM Reference Format:

Anonymous Author(s). 2026. Duet: Orchestrating Buffer Management in Cloud-Native Databases: Full Research Paper. In *Proceedings of ACM Symposium on Cloud Computing (SoCC'26)*. ACM, New York, NY, USA, 13 pages.

1 INTRODUCTION

Cloud-native databases exhibit significant heterogeneity in compute, memory, and storage demands across diverse workloads, with these demands fluctuating independently over time. This variability requires compute, memory, and storage resources to scale independently. To provide such elasticity, modern systems increasingly adopt disaggregated architectures that separate these resources [3, 5, 25]. In such architectures, the memory tier becomes a shared remote buffer for the storage layer, accelerating data access for compute nodes by mitigating remote storage latency [11, 22, 26]. High-bandwidth interconnect technologies such as RDMA [8, 15] and CXL [9, 19] make this remote buffer layer feasible.

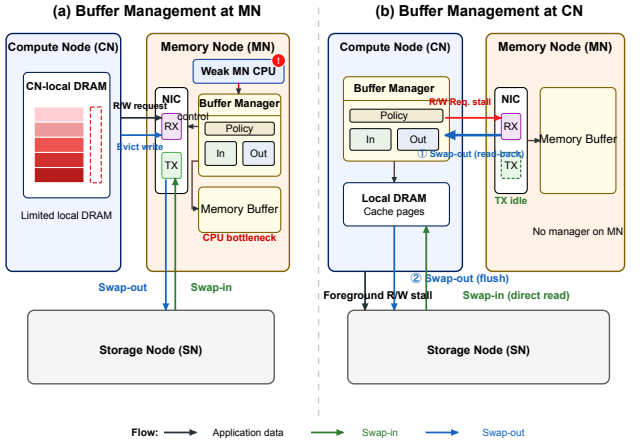


Figure 1: Two performance bottlenecks caused by monolithic buffer management in a disaggregated memory hierarchy. (a) MN-side management runs replacement and swap-in/out at the MN, turning workload-dependent cache-management work into an MN CPU bottleneck. (b) CN-side management moves policy work to the CN but creates bandwidth pressure: dirty pages staged at the MN may be read back before flushing, and CN-to-SN write-back traffic contends with foreground persistence.

However, although disaggregated memory mitigates the latency gap between compute nodes and storage, *existing buffer management architectures fail to scale with the elasticity of disaggregated resources.* A disaggregated buffer manager must coordinate both cache-management decisions and data movement across heterogeneous resource tiers. However, existing designs tightly couple these responsibilities within a single resource domain, creating a fundamental scalability challenge. This challenge manifests as different bottlenecks depending on where buffer management is placed.

First, Figure 1(a) illustrates MN-side buffer management (e.g., PolarDB-MP [29]), where replacement policy execution and swap-in/out are colocated at the MN. As requests from more CNs converge on this monolithic manager, workload-dependent operations such as LRU bookkeeping overload the MN CPU, while the MN must also

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SoCC'26, November 18–20, 2026, Singapore

© 2026 Association for Computing Machinery.

execute bandwidth-bound page movement between memory and storage. The MN's fixed compute capacity therefore cannot scale elastically with transient policy demand from the CNs, making the centralized manager a compute bottleneck and undermining the cost-efficiency of a dense remote memory pool. Crucially, our sensitivity study demonstrates that even with just two CN nodes, executing a monolithic cache management policy at the MN can take milliseconds (§4.5).

Second, Figure 1(b) shows the complementary performance bottleneck of CN-side buffer management. Moving policy execution to the CN uses its abundant compute, but also makes the CN responsible for page movement and write-back, so the background data path competes with foreground traffic. In a simple offloading design [11, 21, 27], dirty pages staged at the MN follow the backflow path highlighted in the figure (MN → CN → SN), adding an MN-to-CN transfer before the page can reach storage [8, 15, 26]. More optimized CN-side designs (e.g., GaussDB-style [18]) retain dirty page images at the CN and avoid this read-back, but the CN-to-SN background write-back path shown in Figure 1(b) remains. It shares the persistence path with foreground durability work such as WAL appends [1, 10]. Under write pressure, this contention produces a *capacity cliff*, after which application execution latency can exceed 500 μs. Thus, even without the extra read-back, CN-side management couples eviction progress to CN-to-SN bandwidth.

Based on the above analysis, the root cause of the disaggregated buffer manager's limited performance scalability becomes clear: traditional cache managers intrinsically combine replacement policies, swap-in, and swap-out into one tightly coupled component, making it impossible to scale efficiently as hardware resources grow. Our empirical profiling reveals a stark contrast among these tasks: while policy execution (86.4%) and foreground swap-ins (8.7%) dominate the CPU overhead, the background swap-out (page staging and storage draining) is bandwidth-bound but CPU-light, contributing merely 4.9% of the CPU cost (§4.5).

To break this fundamental limitation, our idea is to separate background swap-out from replacement policy and foreground swap-in, aligning the three tasks with the hardware tier best suited for their demands. Specifically, we propose to keep replacement policy and foreground swap-in on the CNs (which have abundant CPU and are close to transactions), while offloading asynchronous swap-out to the MNs where it can leverage fixed egress bandwidth to storage. However, physically decoupling buffer management across the network introduces three critical challenges: (1) **decomposing policy, swap-in, and swap-out** to define a clean functional split (CN-side policy/swap-in vs. MN-side swap-out) under hardware asymmetry; (2) **controlling remote policy execution cost** to avoid excessive network round-trips for remote metadata updates; and (3) **coordinating foreground transactions with background page movement** without a centralized lock bottleneck.

To address these challenges, we propose Duet, a novel split buffer-management architecture designed for disaggregated databases. Duet utilizes two specialized engines: **Primo** and **Secondo**. **Primo** runs the control policy and foreground swap-ins on the CNs, making fine-grained eviction decisions and non-blockingly offloading dirty victims to the MN at network speed. **Primo** also reserves pages for eviction and enqueues their identifiers before **Secondo** consumes the queue and asynchronously drains the corresponding MN slots to storage (CN → MN → SN), avoiding data tromboning and foreground interference. Furthermore, to ensure correctness across these decoupled components, we design a lightweight distributed coordination protocol. Using MN-resident metadata and a compact RDMA-atomic `TxLock`, Duet seamlessly coordinates concurrent transaction access, page offloads, and background flushing without a centralized lock bottleneck.

In summary, we make the following contributions:

- We identify two complementary performance bottlenecks of monolithic buffer management in disaggregated databases: MN-side designs suffer an elasticity mismatch, while CN-side designs suffer bandwidth contention.
- We propose Duet, a split buffer-management architecture that aligns each task with its scaling resource. CN-dependent policy work and foreground swap-ins run in **Primo** and scale out with the CNs, while **Secondo** leaves only bandwidth-bound swap-out at the MN, bounding its CPU requirement to the processing rate needed to saturate the MN's maximum storage-egress bandwidth.
- We design a lightweight distributed coordination protocol based on MN-resident metadata and `TxLock`, ensuring correctness across decoupled paths without a centralized lock bottleneck.
- We evaluate Duet on an RDMA cluster against GaussDB-style and PolarDB-style baselines. Across three workloads and five cache settings, Duet improves throughput by approximately 34% and reduces latency by approximately 29% on average relative to the better-performing baseline at each setting.

2 BACKGROUND & MOTIVATION

Traditional buffer management fails in disaggregated environments because the monolithic design of legacy buffer managers clashes with the hardware asymmetry of these systems.

2.1 Disaggregated Memory is Fundamentally Asymmetric

Cloud database workloads do not consume compute, memory, and storage in fixed proportions. Their CPU demand, active working-set size, and persistent-data volume can vary independently across tenants and over time. A conventional server provisions and scales these resources as one unit; responding to a temporary memory increase therefore also adds CPU and storage that may remain idle. This coupling leads either to overprovisioning for peak demand or to insufficient memory when a workload's working set expands. Cloud-native databases have consequently moved toward disaggregated architectures that allow each resource to scale according to its own demand [3, 5, 25].

Disaggregated memory extends this principle beyond compute and storage. RDMA-based systems pool remote DRAM so that cache capacity can grow independently of compute and be reallocated as working sets change [22, 26, 32]. Other systems disaggregate persistent memory for databases [20] or build memory pools over CXL [9, 19]. CXL-based memory tiering further combines local DRAM with device-attached memory and relies on explicit data-placement and migration policies [28, 31]. Network-Attached

Memory systems make the separation between compute and memory servers explicit [4, 30]. A typical deployment connects Compute Nodes (CNs) and Memory Nodes (MNs) through a high-speed RDMA network [8, 15].

The defining characteristic of this architecture is *resource asymmetry*. CNs ship with CPU capacity for query execution and transaction processing but expose relatively small local buffer pools. MNs instead provide dense memory under a fixed CPU budget. The mismatch is not intrinsic MN weakness; MN compute targets bounded memory and I/O service and cannot scale with transient policy demand without recoupling compute to memory capacity [9, 19].

RDMA-capable networks connect these asymmetric nodes, most commonly InfiniBand [8, 11] or RDMA over Converged Ethernet (RoCE) [12]. These fabrics provide high bandwidth and expose one-sided verbs that allow a CN to access registered remote memory without involving the remote CPU in the common data path [15]. RDMA also supports two-sided communication, and the appropriate choice depends on the data structure and transaction protocol [14, 16]. Crucially, these network links are **full-duplex**: receive-side traffic and transmit-side traffic use opposite directions of the NIC fabric. We show that conventional database architectures systematically fail to exploit this bidirectional capacity, leading to severe artificial bottlenecks.

2.2 Monolithic Buffer Management Becomes a Liability

As RDMA makes remote memory fast enough to sit on the data path, the bottleneck shifts from the network itself to the software that orchestrates page movement. In our setting, the large DRAM pool on the Memory Node functions as the **buffer**: it extends the CN's small local DRAM by caching hot pages above the persistent store. The component that manages this pool, the **buffer manager**, originated for a single machine. Classical single-node databases implement the buffer manager as a monolithic, heavy-weight component that ties replacement policy, swap-in, and swap-out into one control structure [10, 23]. Modern buffer managers optimize this monolith aggressively, but still preserve the same unified organization [17]. It acts as a centralized orchestrator with three intertwined tasks:

(1) **Replacement Policy:** Tracking page accesses, maintaining lookup and policy metadata, and selecting victim pages using policies such as LRU or CLOCK.
(2) **Swap-in:** Loading a missing page into the buffer pool and coordinating admission with concurrent page access.
(3) **Swap-out:** Writing dirty victims back to the backing store, freeing buffer frames, and updating the corresponding page-location state.

This monolithic design tightly couples policy execution and swap-in/out execution [10]. When the buffer pool runs out of clean frames, the manager must identify a dirty victim and complete the swap-out work before a new page can be admitted. In a balanced, single-node server, the CPU has the cycles to manage this, and local SSDs absorb the writes. However, when transplanted into a disaggregated environment, this tightly coupled, monolithic nature becomes a severe liability.

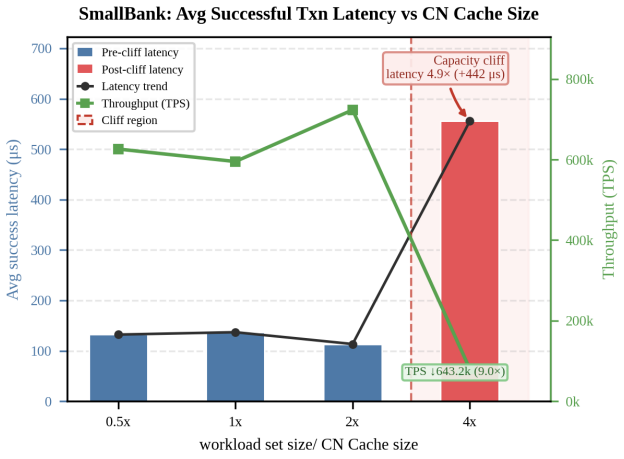


Figure 2: The capacity cliff under CN-local write-back: throughput collapses as the dirty working set exceeds the CN's local DRAM capacity.

2.3 Either Placement Creates a Bottleneck

Deploying a monolithic buffer manager in a disaggregated cluster exposes two complementary placement bottlenecks.

MN-Side Management Creates a Compute Bottleneck. Placing the complete buffer management service at the MN centralizes cache state alongside the large memory pool. Cloud-native databases already move shared storage or memory state outside transaction executors [6, 18, 29]; far-memory systems similarly treat remote memory as a shared resource [11, 21, 22]. However, full buffer management also executes concurrent hash-table lookups, pointer-chasing replacement updates, access tracking, and fine-grained synchronization. These irregular, workload-dependent control tasks quickly saturate the MN's limited CPU. The MN therefore becomes compute-bound before its memory, network, or local storage bandwidth is fully utilized.

CN-Side Management Creates a Bandwidth Bottleneck. Running the service at the CN provides enough compute for policy execution but also makes the CN responsible for storage I/O and page movement. This produces two forms of bandwidth pressure. If the design first stages dirty pages at the MN, the CN must later fetch them back before writing them to storage, creating data backflow along CN → MN → CN → SN. More optimized designs keep dirty pages in CN memory and avoid this backflow, but their background write-back still consumes the same CN-to-storage path used by foreground durability traffic. In both cases, background page movement competes with latency-sensitive foreground traffic.
(1) Remote staging with CN-managed persistence. Far-memory designs can avoid blocking frame reuse by staging dirty victims at the MN [11, 21, 27]. Staging alone does not eliminate the bandwidth bottleneck if the CN still manages final persistence. Each dirty page moves CN → MN during offload, MN → CN when fetched back for flushing, and CN → SN when it reaches storage. The MN-to-CN return leg adds an extra network traversal, while the final CN-to-SN write competes with foreground persistence. This data tromboning

adds network traffic without contributing useful cache capacity or durability [8, 15, 26].

(2) CN-local write-back. GaussDB-style designs keep dirty page images at the CN and flush them directly to the Storage Node [18]. This avoids MN-to-CN data backflow, but places background page write-back on the same CN-to-storage persistence path as foreground WAL appends [1, 10]. Under write-intensive workloads, dirty pages accumulate rapidly in the CN's limited local DRAM. Once the dirty working set exceeds local capacity, the system must continuously flush dirty victims to recycle buffer frames, creating two coupled penalties. First, a transaction that needs a free frame may stall until a dirty victim persists. More importantly, sustained background page flushes contend with foreground WAL appends for the shared persistence bandwidth, inflating commit latency even for transactions that are not directly waiting for frame reclamation. Together, these effects turn memory pressure into a **Capacity Cliff** (Figure 2): performance remains stable while the dirty working set fits in CN memory, but collapses once eviction progress requires sustained storage write-back [5, 18, 20]. On SmallBank, for example, expanding the working set to 4× the CN cache size reduces throughput to approximately 10% of its pre-cliff level and increases average transaction latency by 4.9× (an additional 442 μs).

2.4 Decomposition Breaks the Deadlock

A monolithic buffer manager does not fit the resource asymmetry of disaggregated hardware [2, 11, 22]. Application-integrated far memory [21] and transparent memory offloading [27] further demonstrate the value of separating workload-dependent control from data movement. Running the complete service at the MN creates a compute bottleneck, while running it at the CN creates a bandwidth bottleneck through CN-side write-back or MN-to-CN data backflow. Simply adding MN CPU does not remove this elasticity mismatch: if MN CPU capacity must grow with the number of CNs or their workload-dependent policy demand, MN compute provisioning is no longer determined by the MN's fixed data-movement capacity and instead becomes coupled again to CN workloads.

Resolving this placement dilemma requires decomposing the buffer manager's three tasks. Replacement policy and foreground swap-in remain at the CNs, where policy can use CN compute and swap-in stays close to requesting transactions. Background swap-out uses the MN in two stages. A CN first offloads a dirty page to an MN staging slot, where the page can remain until it is selected for eviction. Primo later reserves a selected staged page and appends its identifier to the Eviction Queue; Secondo then dequeues the identifier and writes the page directly from the MN to storage through the MN's egress path. This draining stage avoids both dirty-page backflow through a CN and background write-back on the CN-to-storage persistence path.

Authoritative page, slot, and ownership metadata remains at the MN because it describes the shared MN-resident staging state and must be accessible to all CNs. To avoid recreating the MN compute bottleneck, CNs access this compact metadata through one-sided and batched RDMA operations and execute workload-dependent policy decisions locally. This policy/swap-in/swap-out decomposition is the core motivation behind Duet and creates the coordination challenges described next.

2.5 Three Challenges in Splitting the Buffer Manager

Splitting buffer management across CNs and MNs requires more than mechanically partitioning an existing buffer pool. The design must first define a clean responsibility boundary and then optimize the control and coordination paths created by that boundary. This leads to three concrete challenges.

C1: Decomposing policy, swap-in, and swap-out under hardware asymmetry. Placing the monolithic manager at one node does not identify the right split. Duet must decide which functions require the CN's CPUs, which functions benefit from the MN's memory capacity and storage-egress path, and what information crosses the boundary between them. The split should keep irregular policy work (access tracking, victim selection, and hot/cold classification) and foreground swap-in at the CN, while leaving only bounded background swap-out—page staging and asynchronous storage I/O—at the MN. At the same time, the two sides need explicit interfaces for metadata, eviction commands, page payloads, and completion events.

C2: Controlling the cost of remote policy execution. After the split, the authoritative page and slot metadata remains at the MN while policy decisions execute at the CN. A direct implementation would fetch metadata and submit decisions one page at a time, adding remote operations and network round trips to every eviction round. Duet must reduce this control-path cost without reconstructing complex policy state at the MN. It therefore needs compact remotely accessible metadata, sampled rather than exhaustive inspection, batched reads and command submission, and a CN-side atomic reservation that rejects stale candidates before submission.

C3: Coordinating foreground transactions with background page movement. Asynchronous backfill, invalidation, and eviction can race with transactions acquiring or releasing page ownership. Independent remote locks can close this race only with an additional atomic operation, a fixed acquisition order, and revalidation or rollback when the two states change concurrently. Duet instead needs one atomic admission boundary that prevents conflicting operations, gives existing transactions priority over slow background movement, and coalesces concurrent I/O without imposing a second remote lock on an uncontended transaction.

3 THE DESIGN OF DUET

3.1 Design Overview and Decomposition

A monolithic buffer manager combines three tasks: replacement policy, swap-in, and swap-out. In a disaggregated system, these tasks favor different resources. Replacement policy involves irregular metadata access and benefits from the stronger CN CPUs, while foreground swap-in benefits from running close to requesting transactions. Background swap-out, including dirty-page staging and storage write-back, benefits from the MN's memory capacity and storage-egress path. Duet therefore decomposes buffer management along these three tasks rather than placing the entire manager at either tier.

Responsibility split. As shown in Figure 3, Duet consists of two collaborating engines. **Primo**, deployed at each CN, executes

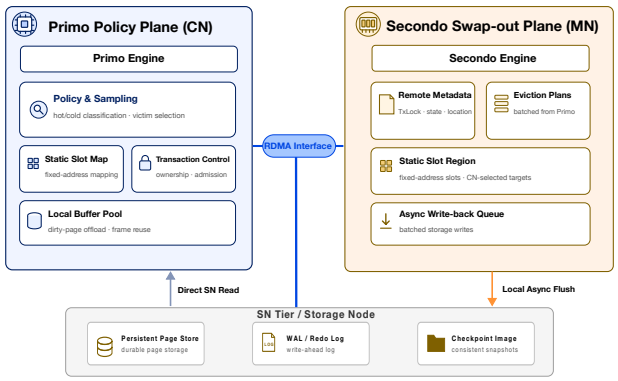


Figure 3: Overview of Duet. Primo executes replacement policy and foreground swap-in at the CNs, stages dirty pages at the MN, and later reserves selected staged pages for eviction. Secondo consumes the resulting queue and executes asynchronous storage I/O for those reserved pages.

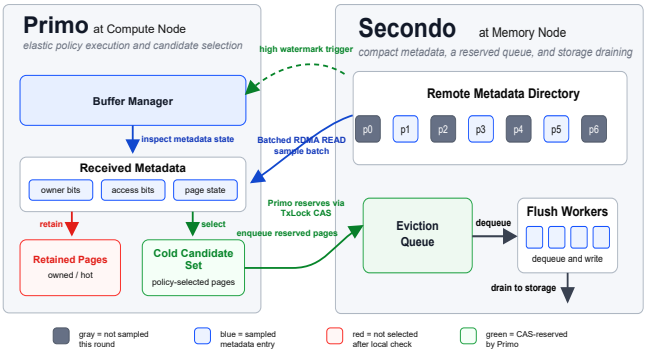


Figure 4: CN-driven sampling in the distributed control path. Primo observes the high watermark, fetches compact metadata samples, filters and ranks candidates locally, atomically reserves eligible pages by installing EVICT, and appends their identifiers to the Eviction Queue. Secondo workers only dequeue reserved identifiers and issue the corresponding flushes.

replacement policy and foreground swap-in: it manages the local buffer pool, tracks transaction-side page use, selects local victims and statically mapped remote slots, and performs policy-intensive global eviction decisions. Primo also reserves staged pages for eviction and enqueues their identifiers. **Secondo**, deployed at the MN, executes background swap-out by consuming this queue and draining the corresponding slots to storage with background I/O workers. This leaves the MN with bounded swap-out work whose maximum rate is constrained by network and storage bandwidth, while policy and swap-in scale with the CN compute tier.

Interfaces. Primo and Secondo interact through three narrow interfaces. First, Primo reads compact entries from the MN's *Remote Metadata Directory* and appends reserved page identifiers to the MN-resident *Eviction Queue* through the control interface. Second, CNs move 4 KB page payloads to or from the *Remote Staging Cache* using one-sided RDMA through the data interface. Third, the shared 8-byte TxLock and lightweight notifications coordinate transaction ownership, page movement, and completion events. The persistent page store, WAL/redo log, and checkpoint image remain in the storage tier. The following sections explain how Duet optimizes the distributed control path, the page data path, and their concurrency boundary.

3.2 Efficient Distributed Control Path

Moving policy execution to the CN creates a distributed control path: the authoritative page and slot metadata resides at the MN, but the CPU-intensive decision logic runs remotely. A naive implementation would read and update this metadata one page at a time, turning each eviction round into a long sequence of network operations. Duet instead makes control proportional to a small sample and communicates decisions in batches.

Compact remote metadata. The Remote Metadata Directory stores a fixed-size entry for each page. An entry contains the 8-byte TxLock, page-state and location fields, and the lightweight access information required by the replacement policy. Keeping entries compact allows Primo workers to fetch groups of candidates with batched RDMA READs rather than traverse pointer-rich policy structures at the MN. The full TxLock format is described in §3.5.

CN-driven sampling. A Primo background worker reads the shared occupancy metadata and starts an eviction round when staging-cache occupancy reaches its high watermark. As shown in Figure 4, it then randomly samples a small batch of directory entries and evaluates ownership, page state, and access frequency locally. It filters out owned or recently accessed pages and ranks the remaining cold candidates using the configured replacement policy. Thus, both eviction triggering and policy bookkeeping consume CN cycles rather than the MN's fixed CPU budget.

Batched decision submission. Because sampling is optimistic, Primo treats each selected candidate as advisory until it reserves the page. It performs one CAS over the complete shared admission word, installing EVICT only if the page still has no owner and no conflicting I/O operation. This CAS is both the final eligibility check and the eviction reservation. A failed CAS returns the current word, allowing Primo to discard or reconsider the stale candidate without a separate read. Primo batches only successfully reserved page identifiers and appends them to the MN-resident Eviction Queue. Because EVICT blocks subsequent ownership acquisition until flush completion, every queued entry remains safe to process. Secondo therefore only dequeues entries and issues their storage writes; it performs no metadata read, eligibility check, or gate acquisition before I/O. All page-location transitions use the same admission word, so the successful Primo CAS serializes eviction with concurrent ownership and page movement. Sampling staleness therefore affects selection efficiency but not correctness, while batching amortizes remote metadata access and queue submission across many candidates.

Partitioned eviction ownership. To avoid duplicate policy work when multiple CNs can assist Secondo, the staging region

is divided into fixed partitions and each partition is assigned to one Primo worker for eviction planning. The partition owner samples candidates, attempts their reservation, and appends successful reservations only for its assigned region, while other CNs operate on different partitions. This partitioning is not a correctness assumption: each candidate remains advisory until its Primo worker successfully reserves it with the page-level `TxLock` CAS. A candidate whose sampled state has become stale simply fails this CAS and is not appended to the Eviction Queue.

Bandwidth-anchored MN provisioning. This organization does more than reduce the cost of each eviction round. Secondo performs neither workload-dependent ranking nor dynamic slot allocation; its steady-state CPU work consists only of consuming reserved entries from the Eviction Queue and issuing storage flushes. The required MN processing rate is therefore bounded by the rate at which its storage-egress path can drain data, while workload-dependent policy work scales with Primo at the CN tier. Queue-driven draining writes reserved pages through the MN-to-storage egress path, without an MN-to-CN read-back leg or use of the CN-to-storage path employed by foreground persistence.

3.3 Decoupled I/O Data Path

The responsibility split is useful only if it also removes storage I/O from the foreground critical path. Duet therefore separates local frame reclamation from final page persistence: Primo offloads a dirty victim at network speed, while Secondo performs the slower storage write-back asynchronously.

Non-blocking dirty-page offload. Primo manages each CN's Local Buffer Pool, Local Page Table, Dirty Victim Queue, and local lock bookkeeping. The MN staging region is statically divided into fixed-address slots whose layout is known to the CNs. Here, *static* refers to the slot-to-address mapping, not to permanent page assignment: Primo still selects and claims a currently free slot using the shared slot metadata. It can then compute the target address locally and transfer the dirty 4 KB payload with a one-sided RDMA WRITE. Only after the payload transfer completes and the directory publishes the new location does Primo reclaim the local frame. The transaction thread therefore does not wait for an SSD write before memory becomes reusable.

Lightweight MN-side draining. Secondo exposes the statically partitioned RDMA region, but it does not run a dynamic slot allocator or initiate eviction rounds. The CPU-intensive slot selection, occupancy inspection, eviction planning, and `EVICT` reservation remain at Primo. Primo appends reserved page identifiers to the Eviction Queue; Secondo workers dequeue them and issue the corresponding writes through the MN-to-SN storage-egress path. This queue-consumer role keeps Secondo's CPU work simple and bounded by the flush rate.

Direct MN-side write-back. A dirty page may remain in an MN staging slot until Primo later selects and reserves it for eviction. Primo then appends the page identifier to the Eviction Queue, and a Secondo worker eventually dequeues the entry and writes the page directly from the MN to the SN through the MN's egress path. Compared with a CN-managed flush of MN-resident pages, this path removes the MN-to-CN read-back leg and avoids consuming the CN-to-storage path used by foreground persistence.

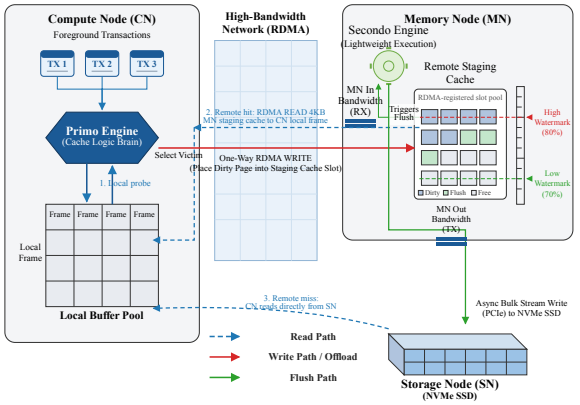


Figure 5: End-to-end read and write workflows. Control operations coordinate page ownership and location, while page payloads move directly between the tier that holds the current image and the requesting or draining node.

3.4 End-to-End Read and Write Workflows

Figure 5 combines the preceding control and data paths into the life cycle of a page.

Read path. A transaction first probes its CN-local buffer pool and enters the page through the local page entry. On a local miss, the CN consults the Remote Metadata Directory and acquires compatible CN-level ownership using the protocol in §3.5. If the page resides in the MN staging cache, the CN fetches the 4 KB payload directly with a one-sided RDMA READ. If the directory indicates a storage-only page, the requesting CN installs `BACKFILL` and fetches the page directly from the Storage Node (SN) into its local buffer. This direct path avoids consuming a Secondo staging slot merely to serve a cold read; the gate prevents concurrent requesters from issuing duplicate storage reads while the fetch is in flight.

Write and offload path. A writer obtains exclusive ownership, updates the page in its local buffer, and commits after its redo records are durable in the distributed WAL [10]. The page remains dirty in CN memory until Primo selects it under memory pressure. Primo then selects and claims a statically mapped staging slot, writes the payload directly to its known address, publishes the new location, and reclaims the CN frame after network completion. These steps decouple frame availability from the final storage write.

Background flush path. When the staging region reaches its high watermark, Primo selects cold pages, atomically installs `EVICT` for each eligible candidate, and appends the reserved page identifiers to the Eviction Queue. Secondo workers dequeue these identifiers and write the corresponding payloads through the local storage interface. After I/O completion, Secondo updates the persistent location state, clears the operation gate, and marks the corresponding static slots free in the shared metadata. Foreground execution interacts with this path only through ownership and admission state, not by performing the storage write itself.

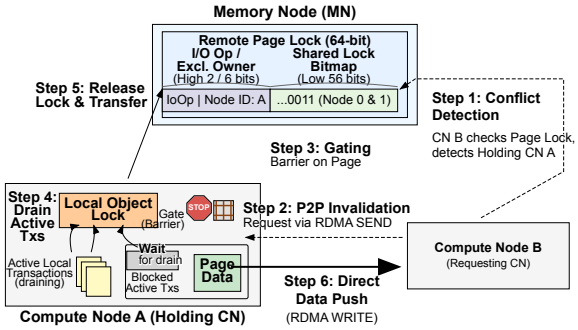


Figure 6: Two-level transaction ownership and the embedded I/O-operation gate. The combined word provides one atomic admission boundary for foreground transactions and background page movement.

3.5 Transaction and I/O Coordination

Disaggregation separates logical transaction ownership from physical page location, but the admission decisions for transaction access and page movement must remain atomic. Independent remote words can provide correctness only through an additional atomic operation plus a protocol that revalidates or rolls back when ownership changes during I/O-gate acquisition. Duet instead combines CN-level ownership and a compact I/O-operation gate in one RDMA-atomic word, making admission and reservation linearize at one CAS.

Two-level ownership. Intra-CN transaction isolation is enforced by local object locks, while inter-CN coordination uses the 8-byte `TxLock` in the Remote Metadata Directory. Its 64 bits comprise a 2-bit I/O-operation field, a 6-bit exclusive-owner field, and a 56-bit shared-owner bitmap at CN granularity. Zero in the exclusive field denotes no exclusive owner; the shared bitmap identifies the CNs holding shared ownership and determines the deployment limit of 56 CNs.

The bitmap is not acquired once per transaction. Multiple local transactions are coalesced behind the CN's local page entry, so the CN installs its shared bit on the first local entrant and releases it after the last local user leaves. A CN acquires shared or exclusive ownership with an RDMA COMPARE_AND_SWAP (CAS) over the complete word. The CAS succeeds only when the observed owners are compatible and the I/O field permits admission. Consequently, an uncontended remote hit needs its ownership CAS but no separate `IOLock` operation.

Embedded I/O-operation gate. The 2-bit field encodes `NONE`, `BACKFILL`, `INVALIDATE`, and `EVICT`. An initiator uses CAS to move from `NONE` to one operation while preserving the ownership fields; completion returns the field to `NONE`. These values are alternative operation states, not a sequential state machine. Because the CAS covers the entire word, exactly one initiator becomes the operation leader, and no new owner can enter between the ownership check and gate installation. For background eviction, the initiating Primo worker installs `EVICT` before it submits the page to Secondo.

Transaction-first conflict policy. A non-`NONE` gate blocks new remote ownership acquisitions but does not preempt transactions that already hold the page. `BACKFILL` serializes an in-flight storage fetch without draining existing holders. `EVICT` can be installed only when both ownership fields are empty; otherwise the eviction attempt backs off and retries later. This policy lets slow page movement wait for foreground use while keeping the normal transaction path free of a second remote lock.

P2P invalidation and local draining. When a CN requests exclusive ownership and observes conflicting owners, it installs `INVALIDATE`, uses the ownership bitmap to identify the holders, and sends targeted invalidation requests. Each holder switches its local page entry to a draining mode: new local transactions stop entering, while current users are allowed to finish. The holder then clears its remote ownership bit and, when necessary, transfers the latest page image directly to the requester with RDMA WRITE. After all holders drain, the requester atomically replaces `INVALIDATE` with its exclusive-owner encoding. The same combined word therefore acts as a compact ownership directory and as the atomic boundary between transaction access and background I/O.

3.6 Durability and Recovery

Duet changes where dirty pages are buffered and when they are propagated to the persistent page store, but it does not redefine the database's durability or log-ordering protocol. We assume that the underlying shared-storage database provides a durable distributed WAL, checkpoints, and a correct per-page redo order, such as the LLSN-based ordering in PolarDB-MP [29]. Following the underlying WAL protocol, redo records may be flushed while a transaction continues executing. The transaction may commit only after all required redo records and its commit status are durable; recovery undoes or ignores records belonging to transactions without a durable commit record [10, 29]. Duet therefore treats CN-local buffers, the MN staging cache, and the Eviction Queue as volatile performance state rather than durability state.

After a transaction releases its page ownership, Primo may stage the dirty page at the MN and reclaim the corresponding CN frame. Completion of this CN-to-MN transfer only confirms that another volatile copy exists; it does not make the page persistent. Because the transaction's redo records are already durable before Primo can reclaim the page, the staged page remains recoverable even if it is lost. Secondo later consumes the Primo-reserved queue entry and propagates the staged page to the persistent page store. It clears `EVICT` and releases the staging slot only after the page-store write completes. The asynchronous write-back path therefore preserves the WAL-before-data ordering while removing the page-store write from the CN's frame-reclamation path.

After a CN failure, surviving staged pages may be reused to accelerate restart, but recovery correctness does not depend on them: missing committed updates can be reconstructed from the latest checkpoint and durable WAL. An MN failure may additionally discard staged pages, queued write-back requests, and the Remote Metadata Directory. During recovery, foreground admission is quiesced, committed page contents are reconstructed according to the underlying database's per-page redo order, and the staging-slot, directory, and operation-gate state is reinitialized before transaction

processing resumes. Thus, Duet introduces no new recovery ordering; it only rebuilds the volatile buffer-management state placed at the MN.

4 EVALUATION

In this section, we evaluate the cache management strategies of **Duet** by comparing them with simplified baselines following the caching principles of **GaussDB** (GaussDB-style) and **PolarDB** (PolarDB-style). Rather than reproducing full industrial distributions, we implement all three systems within a unified transaction execution and storage framework to conduct a controlled comparison. Our evaluation is organized around two levels of evidence. First, we report end-to-end results on three representative OLTP workloads: **SmallBank** [7], **TPC-C** [24], and **TATP** [13]. Second, we conduct four sensitivity studies to isolate why Duet improves performance: latency decomposition under memory pressure, CN-side write-back pressure, cache-management cost decomposition, and an ablation of the integrated TxLock I/O gate. Contention is included only where it helps interpret robustness; it is not the main axis of the evaluation.

4.1 Experimental Setup

Hardware Environment. All our experiments are conducted on a 5-node cluster running Ubuntu 22.04.5 LTS. Specifically, the system topology is configured with 3 Compute Nodes (CNs), 1 Memory Node (MN), and 1 Storage Node. Each machine is equipped with a dual-socket Intel(R) Xeon(R) Platinum 8336C CPU (running at 2.30GHz, totally 128 logical cores), 512 GB of DRAM, and Mellanox ConnectX-5 RDMA Network Interface Cards. The RDMA NICs are connected via an InfiniBand switch to provide high network bandwidth and microsecond-level latency. Although the physical servers are homogeneous, we enforce the disaggregated resource roles in software: CNs use the configured limited local buffer capacity, the MN provides the remote staging pool, and all MN-side components collectively run on two CPU cores. We apply the same two-core MN CPU budget to Duet and the MN-centric baseline. The homogeneous server would otherwise give the MN-centric design substantially more compute than a memory tier provisioned for a fixed CPU budget, so this restriction models the resource asymmetry studied in this paper rather than a hardware-generation advantage.

Implementation and Baselines. To ensure variables are strictly controlled, Duet, GaussDB-style, and PolarDB-style share the same infrastructure. Table 1 summarizes the shared versus differing components across the systems. The comparison purely highlights the impact of cache management.

Workloads. We evaluate the systems using three representative OLTP workloads drawn from standard benchmark suites [7]. **SmallBank** models a banking application with short read-modify-write transactions over checking and savings accounts, making it useful for exposing dirty-page pressure. **TPC-C** is the standard order-processing benchmark with five transaction types and a larger, multi-table working set [24]. **TATP** models a telecommunications subscriber database and is read-dominant, making it a useful counterpoint to the write-heavy SmallBank workload [13].

Table 1: Shared vs. Differing Components across Baselines

Component	Shared?	Notes
Transaction Executor	Yes	Unified simplified framework
Workload Generator	Yes	Same request stream and parameters
RDMA Transport	Yes	Same communication substrate
Storage Interface	Yes	Same SPDK/NVMe path
Basic Lock Model	Yes	Simplified common baseline
Cache Management	No	The main factor under study

Experiment Roadmap. Our main workload experiments compare Duet with GaussDB-style and PolarDB-style baselines on SmallBank, TPC-C, and TATP, each presented in a separate subsection. These experiments answer whether Duet improves end-to-end throughput and latency across different transaction mixes. We then use sensitivity studies to determine whether the improvement comes from removing foreground I/O stalls, moving page write-back off the CN persistence path, moving expensive cache-policy work off the MN, and coordinating transaction ownership with physical page migration through a single atomic admission boundary.

4.2 SmallBank

SmallBank is the primary workload for exposing memory pressure because its update-heavy transactions generate dirty pages quickly. We vary the CN cache capacity and compare throughput and average latency. Duet consistently outperforms both baselines, with the largest gains appearing when the CN cache is constrained. This result indicates that the main benefit comes from removing foreground transactions from the dirty-page eviction and write-back path, rather than from a workload-specific concurrency artifact.

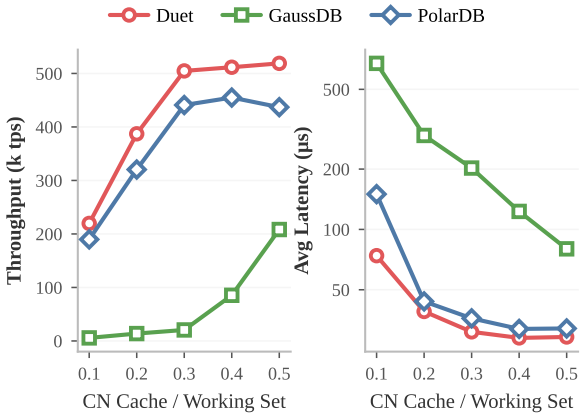


Figure 7: Overall throughput and average latency on SmallBank under different CN cache capacities.

Figure 7 shows that Duet's gains are most pronounced under severe memory pressure. At the smallest CN-cache/workload ratio,

Duet sustains substantially higher throughput than GaussDB-style and remains ahead of PolarDB-style while providing the lowest latency. GaussDB-style exhibits a clear capacity cliff as the CN cache shrinks: synchronous dirty-page flushes increasingly stall foreground transactions and sharply reduce throughput. PolarDB-style is also penalized under severe cache pressure, but for a different reason. A CN-cache miss that hits in the MN cache can be served directly from remote memory without waiting for MN-side request processing. However, when the page also misses in the MN cache, the foreground transaction must wait for the MN to fetch the page from the SN (SSD) before execution can resume. These synchronous storage backfills place SSD access on the transaction's critical path, increasing Non-WAL latency and limiting throughput as cache misses become more frequent. Duet avoids this by staging dirty evictions at the MN and reclaiming CN frames at network speed. Even at the least favorable point, Duet maintains a solid margin over both baselines, confirming that its architectural advantages hold across the full capacity range.

4.3 TPC-C

TPC-C evaluates whether Duet's benefits carry over to a more complex order-processing workload. Unlike SmallBank's short account operations, TPC-C combines five transaction types—New-Order, Payment, Order-Status, Delivery, and Stock-Level—that access multiple logically related tables with different read/write behaviors [24]. New-Order and Payment update frequently accessed warehouse, district, customer, stock, and order records, whereas Order-Status and Stock-Level introduce read-oriented access paths; Delivery additionally touches multiple pending orders. Consequently, the workload combines a larger working set, multi-record transactions, shared metadata, and nonuniform access patterns. It therefore tests both sides of Duet's design: whether dirty-page staging prevents updates from blocking on write-back, and whether the distributed cache hierarchy can serve a diverse access stream without placing MN-side storage backfills on the foreground path.

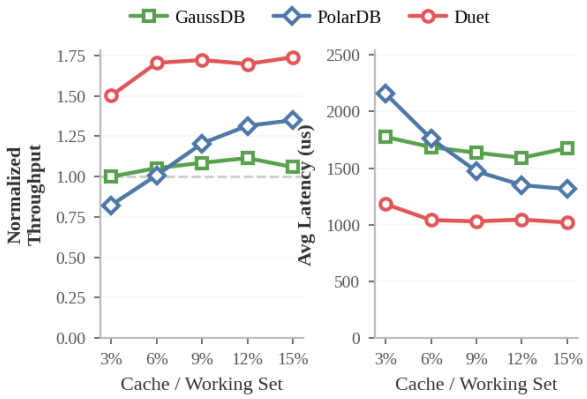


Figure 8: TPC-C capacity scan. Throughput is normalized to GaussDB at the smallest CN-cache setting; latency is reported in absolute terms. Duet maintains the highest throughput and the lowest latency across the full capacity range.

Figure 8 reports throughput and average latency as the ratio between CN-cache capacity and workload size increases from 3% to 15%. Duet sustains a normalized throughput of approximately 1.50–1.74 across the sweep while reducing average latency by more than 30% relative to GaussDB-style at every cache setting. Compared with PolarDB-style, Duet reduces average latency by more than 20% across the full sweep, with the reduction exceeding 40% at the most constrained 3% setting; the gap narrows as the CN cache grows. At the most constrained 3% setting, Duet's advantage over PolarDB-style comes primarily from keeping workload-dependent cache-policy work off the MN. With a large working set and frequent cache misses, PolarDB-style concentrates replacement-policy execution, metadata traversal, and miss handling on the fixed two-core MN, creating a cache-management computation bottleneck. As more pages fit in the CN cache, PolarDB-style invokes these MN-side management paths less often and therefore narrows the gap. In contrast, Duet maintains a stable advantage over GaussDB-style across the entire sweep because TPC-C is write-heavy: staging dirty pages at the MN removes dirty-page eviction and storage write-back from the CN's foreground persistence path, allowing local frames to be reclaimed at network speed while MN-side draining proceeds asynchronously. The result extends the SmallBank finding to richer transaction logic and multi-table accesses, while distinguishing Duet's benefits over the two baselines: lower MN-side cache-management overhead than PolarDB-style, and a decoupled dirty-data path compared with GaussDB-style.

4.4 TATP

TATP evaluates Duet under a read-dominant telecommunications workload. Its subscriber records are spread across the Subscriber, AccessInfo, SpecialFacility, and CallForwarding relations, and its transaction mix is dominated by short indexed lookups, with a smaller fraction of updates to subscriber locations and call-forwarding state [13]. Compared with SmallBank, TATP creates less dirty-page pressure and shifts the primary stress from write-back to read misses and page placement. It is therefore an important counterpoint: improvements on TATP cannot be explained solely by accelerating dirty-page eviction, but must also reflect how effectively the system uses the combined CN–MN memory hierarchy and handles pages that are absent from the CN cache.

As shown in Figure 9, TATP's read-dominant transaction mix generates relatively little dirty-page pressure, shifting the source of Duet's benefit from dirty-page write-back primarily to miss handling and cache-management overhead. The miss-path benefit is most apparent when the CN cache is small and all three systems frequently leave the local-cache fast path. For GaussDB-style, foreground transactions can be delayed by synchronization with background page flushing when local frames must be reclaimed. For PolarDB-style, a page absent from both the CN and MN caches requires the foreground transaction to wait for the MN to read it from the SSD, exposing a longer CN–MN–SSD data path. Duet mitigates rather than eliminates the cold-read cost: when a page resides only at the storage tier, the requesting CN fetches it directly from the SN, avoiding the MN-mediated backfill path, while the embedded `BACKFILL` gate coalesces concurrent misses and suppresses duplicate SSD reads. As the CN cache grows, evictions and storage

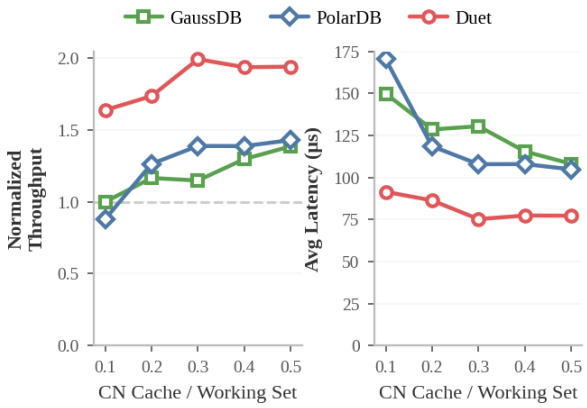


Figure 9: TATP capacity scan. Throughput is normalized to GaussDB at the smallest CN-cache setting; latency is reported in absolute terms. Duet benefits most directly from shorter miss handling under tight CN-cache capacity and retains a clear advantage as the cache grows.

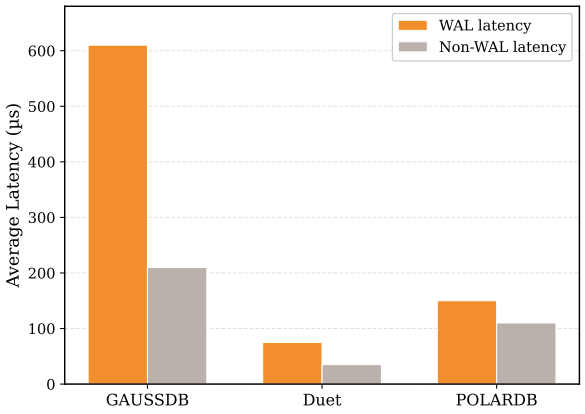


Figure 10: WAL and Non-WAL latency under constrained CN cache capacity. The two paths progress concurrently and are therefore shown separately rather than stacked. GaussDB-style suffers from severe persistence contention, inflating its WAL latency.

misses become less frequent, so the contribution of these miss-path optimizations decreases. Nevertheless, Figure 9 shows that Duet retains a clear throughput and latency advantage at larger cache settings, reflecting the combined effect of direct cold reads, lighter MN-side cache management, and integrated I/O coordination rather than dirty-page staging alone.

Across these three workloads and five cache settings, we compute the improvement at each point relative to the better-performing baseline for that metric—the baseline with higher throughput or lower latency—and then take the arithmetic mean over all 15 points. Under this conservative comparison, Duet improves throughput by approximately 34% and reduces latency by approximately 29% on average.

4.5 Sensitivity Study

The main workload results show the end-to-end effect of Duet. We next run four sensitivity studies to explain where the improvement comes from and when it becomes smaller. Unless otherwise stated, these experiments use SmallBank because it exposes dirty-page pressure clearly.

Latency Breakdown under Memory Pressure. We first separate transaction latency under constrained CN cache capacity into two concurrently progressing paths. This experiment complements the capacity-cliff result by showing which path dominates the critical path and isolating the impact of persistence contention. We independently measure WAL latency (foreground persistence) and Non-WAL latency (execution, lock wait, remote fetch, etc.). Because WAL flushing overlaps with transaction execution, these measurements are shown as separate bars and must not be added.

The result in Figure 10 exposes the flaw of coupled write-back in GaussDB-style designs. Because background cleaning shares the same persistence path as foreground WAL appends, its WAL latency is more than 8× that of Duet and more than 4× that of PolarDB-style; its Non-WAL latency is also more than 5× that of Duet. PolarDB-style avoids this write-back contention, but its MN-centric management leaves its Non-WAL latency more than 3× that of Duet. In contrast, Duet decouples dirty-page write-back from the foreground persistence path while keeping control logic off the MN, yielding the shortest WAL and Non-WAL paths. These paired measurements are not additive; they identify which concurrent path determines the observed transaction latency.

GaussDB-style CN-side Write-back Pressure. We then measure how CN cache capacity affects the volume of dirty-page data flushed by GaussDB-style CNs to storage. This metric counts CN-originated page write-back traffic rather than the total amount of data eventually written to the SSD. We normalize each result to GaussDB-style at a CN-cache/workload ratio of 0.5, thereby comparing the baseline against itself as memory pressure increases.

Figure 11 shows that GaussDB-style CN-side write-back pressure is largest when memory pressure is most severe. Relative to its own write-back volume at a CN-cache/workload ratio of 0.5, GaussDB-style flushes 57× and 26× as much data at ratios of 0.1 and 0.2, respectively. Once the ratio reaches 0.5, fewer dirty pages are forced out of the CN cache, so the CN-side write-back volume returns to the normalized baseline. This self-comparison exposes the mechanism behind GaussDB-style's capacity cliff: constrained CN cache capacity sharply increases background page-flush traffic on the same persistence path used by foreground transactions.

Cache-Management Cost Decomposition. We next quantify how expensive it is to run a complete cache-management policy at the Memory Node—as a centralized or MN-centric buffer manager would—and identify which component dominates the cost. To answer this, we decompose the total cache management cost into three components: **Swap-out I/O** (physical write-back of evicted pages), **Swap-in I/O** (loading pages from storage), and **Policy** (LRU bookkeeping, metadata traversal, and hit/miss checks). We simulate

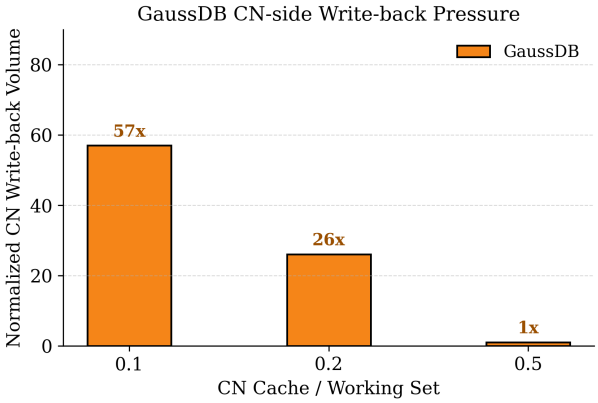


Figure 11: GaussDB-style CN-side write-back volume at CN-cache/workload ratios of 0.1, 0.2, and 0.5, normalized to its own volume at 0.5. Constrained CN cache capacity increases the flushed data volume by 57× and 26× at ratios of 0.1 and 0.2, respectively.

a centralized cache-management workload with 10,000 pages, a 2,000-page buffer pool, 50,000 accesses per CN, and a Zipf-1.2 access distribution, sweeping the number of CNs from 1 to 4. This experiment is a controlled execution-time decomposition of a centralized policy rather than an online measurement of MN CPU utilization in Duet.

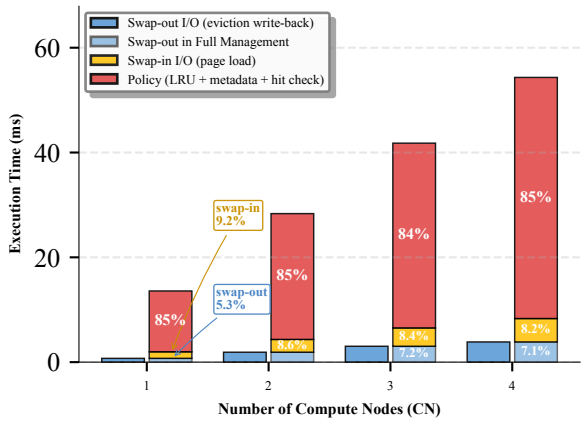


Figure 12: Decomposition of cache management overhead into Swap-out I/O, Swap-in I/O, and Policy (LRU + metadata + hit check). Swap-out is only 5–7% of total cost, while Policy execution dominates at 85–86%.

Figure 12 reveals a striking result: across all CN counts, **Swap-out I/O accounts for only 4.9%–7.1% of total execution time**, while **Policy execution dominates at 85.0%–86.4%**. Swap-in I/O contributes the remaining 7.2%–8.7%. This means that a Memory Node running a complete buffer-management policy would spend the vast majority of its cache-management time on LRU bookkeeping, metadata pointer-chasing, and hash-table lookups—not on the I/O operations that benefit from the MN's proximity to storage.

This decomposition provides the quantitative foundation for Duet's architectural split. By placing the dominant Policy work and foreground Swap-in path on the CNs (Primo Engine), while leaving only lightweight batched Swap-out I/O on the MN (Secondo Engine), Duet retains only the 4.9%–7.1% component at the MN in this decomposition. The decomposition therefore isolates the computational rationale for Duet's responsibility split independently of runtime scheduling effects.

I/O-Gate Ablation. Finally, we isolate the benefit of embedding the I/O-operation gate in `TxLock`. The ablated variant preserves the same data path but uses a separate `IOLock`, requiring one CAS for transaction ownership and another for I/O coordination. It therefore loses the single-word atomic admission boundary and cannot coalesce concurrent page movement as efficiently as full Duet. When most accesses hit locally and neither contention nor storage misses exercise the gate, full Duet improves throughput by 2%, mainly by eliminating the second CAS. Under concurrent misses to a hot page, `BACKFILL` elects one storage-read leader and suppresses duplicate SSD reads, yielding 50% higher throughput. Frequent invalidation and eviction similarly expose the value of atomic gating: the `INVALIDATE` and `EVICT` states reduce transaction stalls during page movement and improve throughput by 30% under high write contention. With the default mixed-OLTP parameters, these effects combine to produce an overall throughput improvement of approximately 39%. Thus, the embedded gate is not merely a metadata-packing optimization: its largest benefit comes from coalescing slow I/O and preventing page movement from repeatedly stalling transactions.

4.6 Discussion

We note a few limitations to carefully contextualize our evaluation. First, our baselines are simplified implementations reflecting specific caching philosophies, rather than exact replicas of full-featured industrial databases. Second, because we use a unified concurrency control framework, the bottleneck observed under high contention largely reflects the limitations of generalized lock management rather than the upper limits of highly customized industrial engineering. Third, our controlled baselines intentionally isolate cache-management choices; they should be read as architectural comparisons rather than product-to-product benchmarks. Ultimately, these results confirm that cache management is a first-order performance factor in disaggregated databases. The latency measurements and the GaussDB-style self-comparison show how coupled write-back inflates foreground persistence latency and CN-side flush traffic under memory pressure, while the cost decomposition supports moving cache-policy work away from the MN and the I/O-gate ablation quantifies the benefit of integrated coordination. Under heavy conflicts, the bottleneck can shift from caching to concurrency coordination, but Duet remains competitive because it has already removed the structural I/O stalls that dominate under memory and write-back pressure.

5 CONCLUSION

Disaggregated databases cannot treat buffer management as a single resource class: replacement policy, swap-in, and swap-out place different demands on asymmetric compute and memory tiers. We presented DUET, which separates these three tasks across two engines. **Primo** executes policy, slot selection, and foreground swap-in at the CNs, stages dirty pages at the MN, and later reserves selected staged pages for eviction. **Secondo** consumes the resulting queue and asynchronously writes those reserved pages to storage. Compact MN-resident metadata, batched advisory eviction, and the integrated TxLock admission word coordinate the resulting distributed control and data paths without moving workload-dependent replacement policy back to the MN.

Across three workloads and five cache settings, our RDMA-cluster evaluation shows that this split improves throughput by approximately 34% and reduces latency by approximately 29% on average relative to the better-performing baseline at each setting, with the largest gains under memory and write-back pressure. The sensitivity studies further connect these gains to reduced persistence contention, the growth of GaussDB-style CN-side write-back pressure under constrained cache capacity, the dominance of policy work in cache-management cost, and atomic coordination of foreground access with page movement. These results demonstrate that aligning buffer-management responsibilities with their scaling resources can avoid the compute and bandwidth bottlenecks created by monolithic placement.

REFERENCES

[1] Marcos K. Aguilera, Nadav Amit, Irina Calciu, Xavier Deguillard, Jayneel Gandhi, Stanko Novaković, Arun Ramanathan, Pratap Subrahmanyam, Lalith Suresh, Kiran Tati, Rajesh Venkatasubramanian, and Michael Wei. 2018. Remote Regions: A Simple Abstraction for Remote Memory. In *Proceedings of the 2018 USENIX Annual Technical Conference*. USENIX Association, 775–787.

[2] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can Far Memory Improve Job Throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*. ACM, 1–16.

[3] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravella, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. 2019. Socrates: The New SQL Server in the Cloud. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, 1743–1756.

[4] Carsten Binnig, Andrew Crotty, Alex Galakatos, Tim Kraska, and Erfan Zamanian. 2016. The End of Slow Networks: It's Time for a Redesign. *Proceedings of the VLDB Endowment* 9, 7 (2016), 528–539.

[5] Wei Cao, Yingqiang Zhang, Xinjun Yang, Feifei Li, Sheng Wang, Qingda Hu, Xuntao Cheng, Zongzhi Chen, Zhenjun Liu, Jing Fang, Bo Wang, Yuhui Wang, Haiqing Sun, Ze Yang, Zhushi Cheng, Sen Chen, Jian Wu, Wei Hu, Jianwei Zhao, Yusong Gao, Songlu Cai, Yunyang Zhang, and Jiawang Tong. 2021. PolarDB Serverless: A Cloud Native Database for Disaggregated Data Centers. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, 2477–2489.

[6] Alex Depoutovitch, Chong Chen, Per-Ake Larson, Jack Ng, Shu Lin, Guanzhu Xiong, Paul Lee, Emad Boctor, Samiao Ren, Lengdong Wu, et al. 2023. Taurus MM: bringing multi-master to the cloud. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3488–3500.

[7] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proceedings of the VLDB Endowment* 7, 4 (2013), 277–288.

[8] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. 2014. FaRM: Fast Remote Memory. In *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 401–414.

[9] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct Access, High-Performance Memory Disaggregation with DirectCXL. In *Proceedings of the 2022 USENIX Annual Technical Conference*. USENIX Association, 287–294.

[10] Jim Gray and Andreas Reuter. 1993. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

[11] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G Shin. 2017. Efficient Memory Disaggregation with Infiniswap. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 649–667.

[12] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over Commodity Ethernet at Scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*. ACM, 202–215.

[13] IBM Software Group. 2009. *Telecommunication Application Transaction Processing Benchmark Description*. Available at https://tatpbenchmark.sourceforge.net/.

[14] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA Efficiently for Key-Value Services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*. ACM, 295–306.

[15] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design Guidelines for High Performance RDMA Systems. In *Proceedings of the 2016 USENIX Annual Technical Conference*. USENIX Association, 437–450.

[16] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. FaSST: Fast, Scalable and Simple Distributed Transactions with Two-Sided (RDMA) Datagram RPCs. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 185–201.

[17] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. 2018. LeanStore: In-Memory Data Management Beyond Main Memory. In *Proceedings of the 2018 IEEE 34th International Conference on Data Engineering*. IEEE, 185–196.

[18] Guoliang Li, Wengang Tian, Jinyu Zhang, Ronen Grosman, Zongchao Liu, and Sihao Li. 2024. GaussDB: A Cloud-Native Multi-Primary Database with Compute-Memory-Storage Disaggregation. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3786–3798.

[19] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Dan Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 574–587.

[20] Chaoyi Ruan, Yingqiang Zhang, Chao Bi, Xiaosong Ma, Hao Chen, Feifei Li, Xinjun Yang, Cheng Li, Ashraf Aboulnaga, and Yinlong Xu. 2023. Persistent Memory Disaggregation for Cloud-Native Relational Databases. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, 498–512.

[21] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K. Aguilera, and Adam Belay. 2020. AIFM: High-Performance, Application-Integrated Far Memory. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 315–332.

[22] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 69–87.

[23] Michael Stonebraker. 1981. Operating System Support for Database Management. *Commun. ACM* 24, 7 (1981), 412–418.

[24] Transaction Processing Performance Council. 1992. *TPC Benchmark C: Standard Specification*. Available at https://www.tpc.org/tpcc/.

[25] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 1041–1052.

[26] Ruihong Wang, Jianguo Wang, Stratos Idreos, M Tamer Özsu, and Walid G Aref. 2022. The Case for Distributed Shared-Memory Databases with RDMA-Enabled Memory Disaggregation. *Proceedings of the VLDB Endowment* 16, 1 (2022), 15–22.

[27] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. 2022. TMO: Transparent Memory Offloading in Datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 609–621.

[28] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. Nomad: Non-Exclusive Memory Tiering via Transactional Page Migration. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 19–35.

[29] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Bo Wang, Jing Fang, Chuan Sun, and Yuhui Wang. 2024. PolarDB-MP: a multi-primary cloud-native database via disaggregated shared memory. In *Companion of the 2024 International Conference on Management of Data*. 295–308.

[30] Erfan Zamanian, Carsten Binnig, Tim Harris, and Tim Kraska. 2017. The End of a Myth: Distributed Transactions Can Scale. *Proceedings of the VLDB Endowment* 10, 6 (2017), 685–696.

[31] Yuhong Zhong, Daniel S. Berger, Carl Waldspurger, Ryan Wee, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D. Hill, Mosharaf Chowdhury,

and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 37–56.

[32] Yijie Zhong, Minqiang Zhou, Zhirong Shen, and Jiwu Shu. 2024. UniMem: Redesigning Disaggregated Memory within A Unified Local-Remote Memory Hierarchy. In *Proceedings of the 2024 USENIX Annual Technical Conference*. USENIX Association, 463–477.